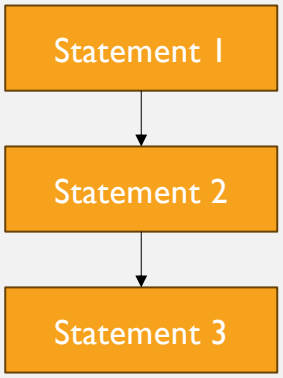
DECISIONS (IF/THEN/ELSE)

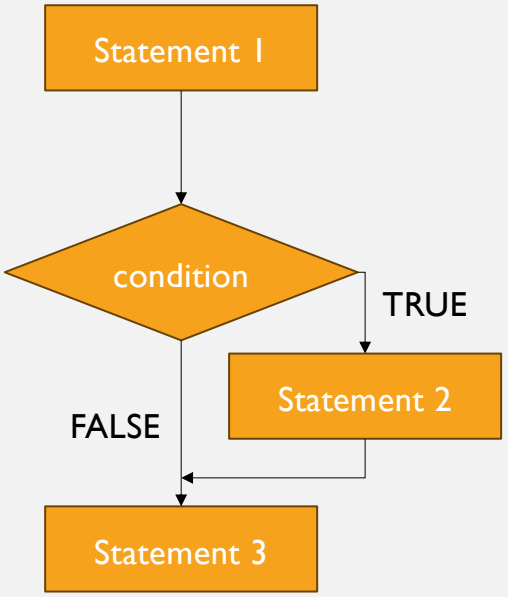
CS1910

Computer Science I

FLOW OF CONTROL

- Typically the flow of our programs has been a sequence of statements executed in order
- There are statements that alter this order:
  - Decision Statements – things that go one way based on whether something is true, and another way if it is false
  - Repetition Statements – things that happened over and over as long as something is true, and stops if something is false
- Both require a *conditional statement* to change the flow of our program from being just one step after another







- You're going to a bank machine to withdraw some money
- With your partner, answer:
  - What are you trying to achieve?
  - What steps do you take, in order?
  - What might stop you from getting your money?
- Pick one thing that could stop you from getting your money and phrase it as:
  - "If X is true, then Y"

(REFINED) QUESTIONS TO ASK

1. What are we trying to achieve?
2. What do we know?
3. What information is missing?
4. What kind of information is each piece?
5. Can we break it down into smaller parts?
6. What decisions need to be made?
7. What are the patterns? Have we solved it before?
8. Can we represent it differently?
9. How will we know it works?

WITHDRAWING MONEY ANALYSIS

- Start with the goal: *What are we trying to achieve?*
  - **Decomposition**: *Can we break it down into smaller parts?*
  - **Pattern recognition**: *What are the patterns? Have we solved it before?*
  - **Abstraction**: *What do we know? What's missing? What kind of information is each piece?*
  - **Algorithm design**: *What decisions need to be made? Can we represent it differently?*
- Finish by checking (**Testing**): *How will we know it works?*



THE GOAL
ASK: WHAT ARE WE TRYING TO ACHIEVE?

- Get the cash you asked for
- But only if:
  - It's really you
  - The money is really there
- The machine has two goals:
  - help you, and
  - protect your account



DECOMPOSITION

ASK: CAN WE BREAK IT DOWN INTO SMALLER PARTS?

- Insert card
- Enter PIN
- Choose account (chequing / savings)
- Enter amount
- Machine checks the request
- Dispense cash
- Print receipt, return card

- **Did your list match? What did you add or leave out?**

PATTERN RECOGNITION

ASK: WHAT ARE THE PATTERNS? HAVE WE SOLVED IT BEFORE?

- Where does the machine *check* something?
  - Is the PIN correct?
  - Is there enough money in the account?
  - Is the amount under the daily limit?
  - Does the machine have enough cash?
  - Is the amount a multiple of $20?
- The pattern: **check a condition then choose what happens next**

- **Where else have you seen this pattern in your life? Where have you see it this morning?**

ABSTRACTION

ASK: WHAT DO WE KNOW? WHAT INFORMATION IS MISSING?

- What the bank already knows: card number, PIN, account balance, daily limit, cash in machine
- What's missing? - your PIN entry, which account, the amount
- What it keeps track of as you go: number of PIN attempts

- Abstraction is as much about hiding information you don't need as it is defining what you do need
  - What it doesn't need: your name, the colour of your card, why you want the money



WHAT KIND OF INFORMATION?
ASK: WHAT KIND OF INFORMATION IS EACH PIECE?

- Card number → text (string); it identifies you, we never do math with it
- PIN → text (string); why? Think about a PIN of 0042 – as an int we lose the 00
- Balance, amount, daily limit → numbers with decimals (float)
- PIN attempts → whole number (integer)
- PIN correct? Enough money? → True or False (Boolean)

- **Tip: ask "what will we *do* with it?" to decide its type**

ORDER AND COMBINATIONS MATTER
ASK: WHAT DECISIONS NEED TO BE MADE, AND IN WHAT ORDER?

- Some checks must come first
  - Don't check the balance until the PIN is correct
- Some decisions need several conditions to all be true
  - If amount ≤ balance **and** amount ≤ daily limit **and** amount ≤ cash in machine, then dispense
- Some decisions lead to other decisions
  - If the PIN is wrong three times, then keep the card

- However – checking the PIN multiple times requires repetitions– so we will assume for now ANY error leads to returning the card
- This makes for a very **rude** bank machine!

ALGORITHM DESIGN
ASK: CAN WE REPRESENT IT DIFFERENTLY?

- Read the card
- Ask for the PIN
- If the PIN is correct ask for amount, otherwise return card
- Ask for the amount
  - If the amount is not a multiple of $20 show an error and return card
  - If the amount is more than the balance, daily limit or cash available, show an error and return card
  - Otherwise, dispense cash, subtract from balance, print receipt, return card

TEST YOUR ALGORITHM
ASK: HOW WILL WE KNOW IT WORKS?

- **Find a Partner:** Walk through your steps with these cases:
  - Wrong PIN?
  - Balance $100, request $100
  - Balance $100, request $120
  - Request $30
  - Request $0
- Did your algorithm do the right thing every time?
- The ***edge cases*** (exactly equal, zero, the third try) are where bugs hide

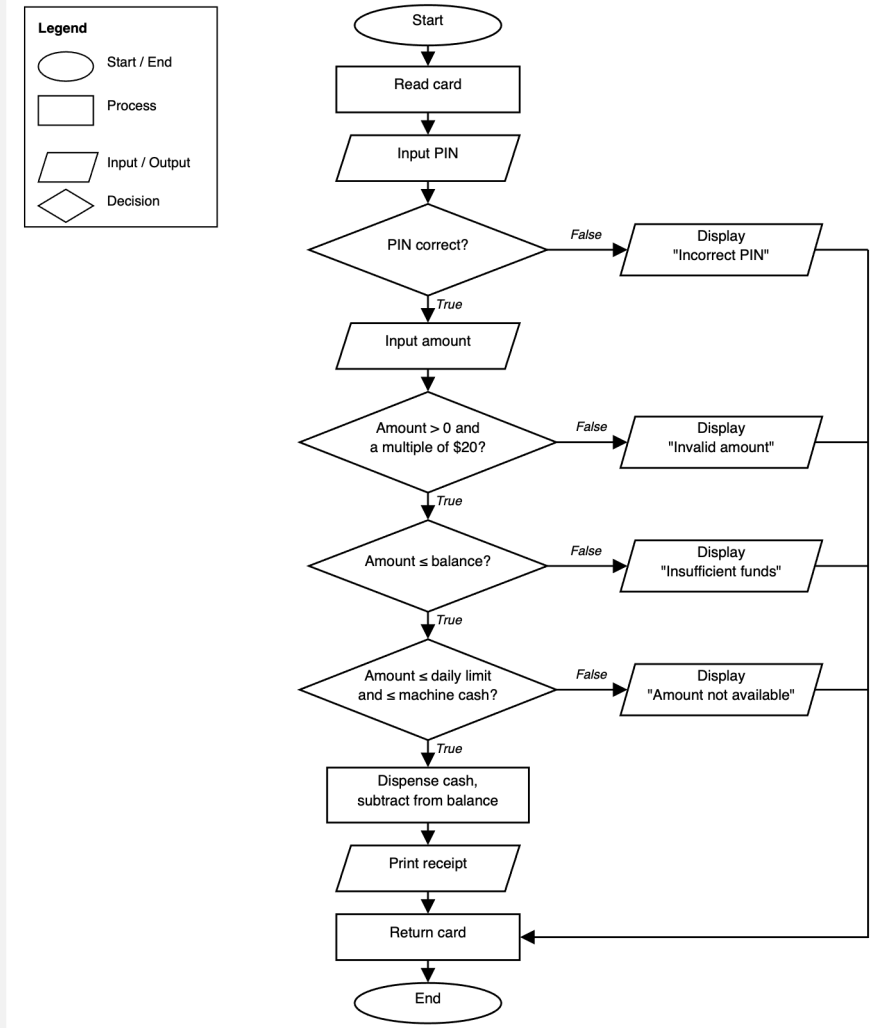
- Read the card
- Ask for the PIN
- If the PIN is right ask for amount, otherwise display "Incorrect PIN" and return the card.
- Ask for the amount
- If the amount is not a multiple of $20, show an error and return card
- If the amount is more than the balance, daily limit or cash available, show an error and stop
- Otherwise, dispense cash, subtract from balance, print receipt, return card

WHERE WE'RE GOING NEXT

ASK: CAN WE REPRESENT IT DIFFERENTLY?

- Our plain-English steps become:
  - **Pseudocode**: structured steps, close to code
  - **Flowchart**: decisions drawn as diamonds, with True and False paths
  - **Python Syntax**: if, elif, else
- The nine questions work for every problem this term, not just bank machines

ALGORITHM AS FLOW CHART



ALGORITHM AS PSEUDOCODE

```
START
READ card
INPUT pin_entry
IF pin_entry = stored_pin THEN
    INPUT amount
    IF amount > 0 AND amount is a multiple of 20 THEN
        IF amount <= balance THEN
            IF amount <= daily_limit AND amount <= machine_cash THEN
                DISPENSE cash
                SET balance TO balance - amount
                PRINT receipt
END
ELSE
                DISPLAY "Amount not available"
            END IF
        ELSE
            DISPLAY "Insufficient funds"
        END IF
    ELSE
        DISPLAY "Invalid amount"
    END IF
ELSE
    DISPLAY "Incorrect PIN"
END IF
RETURN card
```

CODE EXAMPLES

- https://colab.research.google.com/github/UPEI-CS/CS1910-Students/blob/main/lectures/06-Decisions/CS1910_Decisions_Workbook.ipynb

BOOLEAN EXPRESSIONS

- Recall that Boolean values are either True or False
- Boolean expressions evaluate to a Boolean value
  - amount <= balance → True or False
- **Relational (comparison) operators:** ==, !=, >, <, >=, <=
  - Note: == compares, = assigns
- **Logical operators:** and, or, not
  - and → True only if both sides are True
  - or → True if at least one side is True
  - not → flips True to False, and False to True

BOOLEAN EXPRESSIONS

- **Order of precedence:** BEDMAS, then comparisons, then logic operators
  - Brackets ( )
  - Exponents **
  - Division and Multiplication / // % *
  - Addition and Subtraction + -
  - Comparisons == != > < >= <=
  - not
  - and
  - or

- Example:

  SET balance to 100

  SET amount to 60:

  balance - 20 >= amount **and** amount > 0

  80 >= 60 **and** 60 > 0

  True **and** True

  True

QUICK HITTERS

- What is the result of the expression

```
42 != 42
42 <= 42
“A” < “A” or “BB” < “BBB”
1 - 1 > 0 and -2 > 0
len(“ABC”) > 0 and not len(“ABC”) < 2
```

*Logical operators have a lower precedence than Relational operators

CONDITIONAL STATEMENTS

- Uses a Boolean expression to determine which statements are executed next (or not executed at all)
- Programmers use these to program decision logic
- Main types of Conditional Statements:

if Statement	if-else Statement	If-elif-else Statement

IF STATEMENT

Code that runs when the condition is True

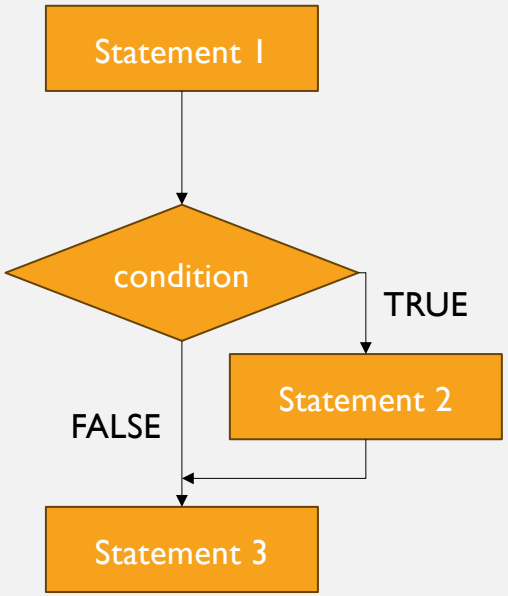
Must be indented in a code block

Boolean expression or value

```
if condition:
    # if block (indented)

# code after else block
```

End of indentation indicates end of the if block



EXAMPLE

```
# if statement

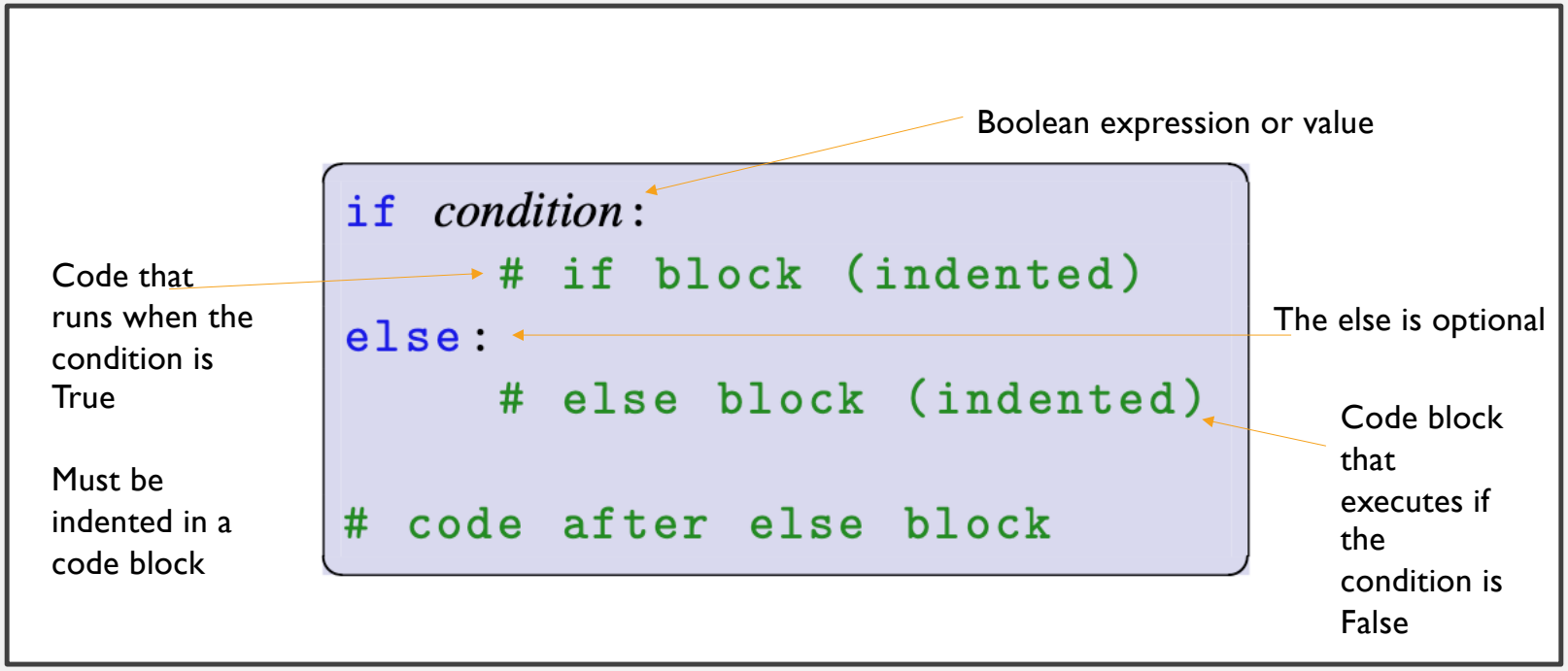
bonus = 6
grade = 72

if bonus > 5:
    grade += bonus

print(f"grade: {grade}")
```

```
grade: 78
```

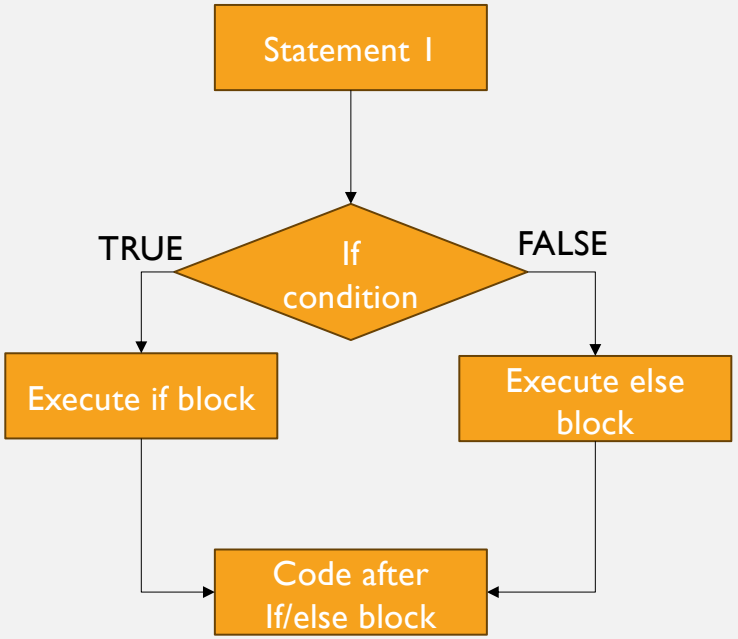
IF-ELSE STATEMENT



IF-ELSE STATEMENT

```
if condition:
      # if block (indented)
else:
      # else block (indented)

# code after else block
```



SELF-CHECK

- We want to detect that someone hasn't rolled double 6's:

```
# not double 6's
die_roll1 = 5
die_roll2 = 6

if not die_roll1 ==6 and not die_roll2 == 6:
    print("not 12 the hard way")
else:
    print("ack double 6")
```



What's the output?

SELF-CHECK

- We want to detect that someone hasn't rolled double 6's:

```
# not double 6's
die_roll1 = 5
die_roll2 = 6

if not die_roll1 ==6 and not die_roll2 == 6:
    print("not 12 the hard way")
else:
    print("ack double 6")
```



ack double 6 ← Why?

Can you fix the logic?

```python
#  if - else statement

shoe_size = 7
price = 50

if shoe_size < 5:
    #childrens clothing tax
    tax = 1.09
else:
    tax = 1.15

price *= tax

print(f"tax: {tax:.2f}, price: {price:.2f}")
```

```
tax: 1.15, price: 57.50
```

EXAMPLE

IF-ELIF-ELSE STATEMENT

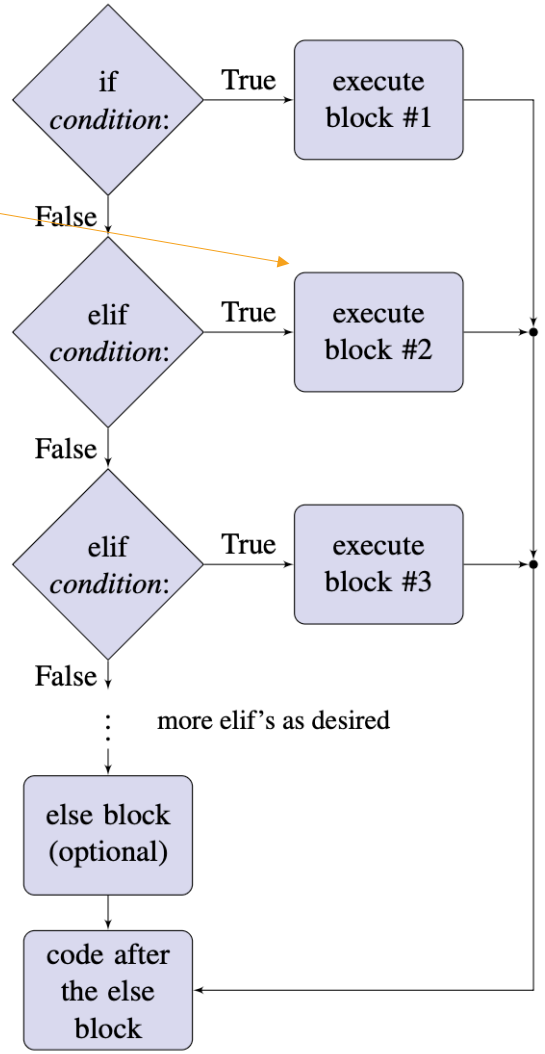
Only one of these code blocks is executed

```
if condition:
      # block 1 (indented)
elif condition:
      # block 2 (indented)
elif condition:
      # block 3 (indented)

# ... more elif's as desired

else:  # (optional)
      # else block

# code after the else block
```



EXAMPLE

```
# if-elif-else statement

grade = 78
letter_grade = ""

if grade > 90:
    letter_grade = "A"
elif grade > 75:
    letter_grade = "B"
elif grade > 60:
    letter_grade = "C"
elif grade > 50:
    letter_grade = "D"
else:
    letter_grade = "F"

print(f"grade: {grade} ({letter_grade})")
```

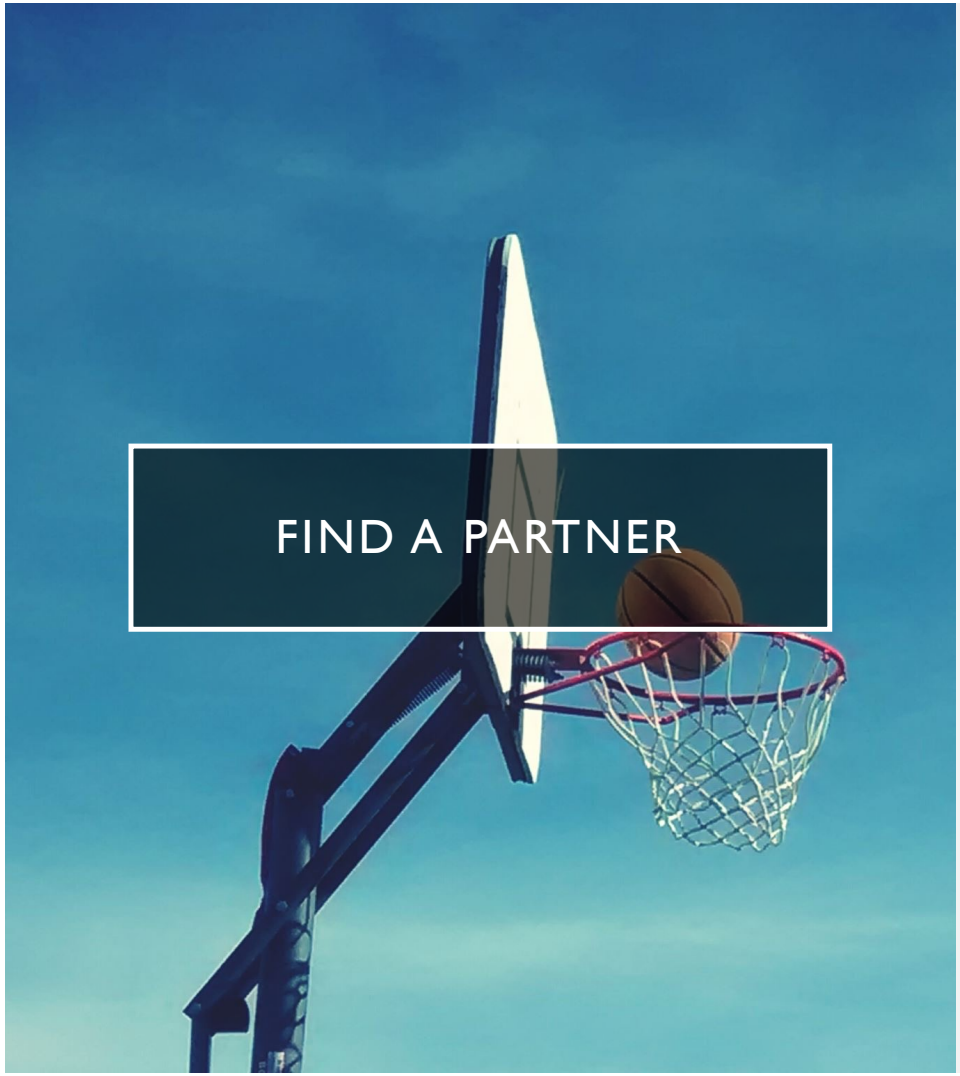
```
grade: 78 (B)
```

NESTED CONDITIONAL STATEMENTS

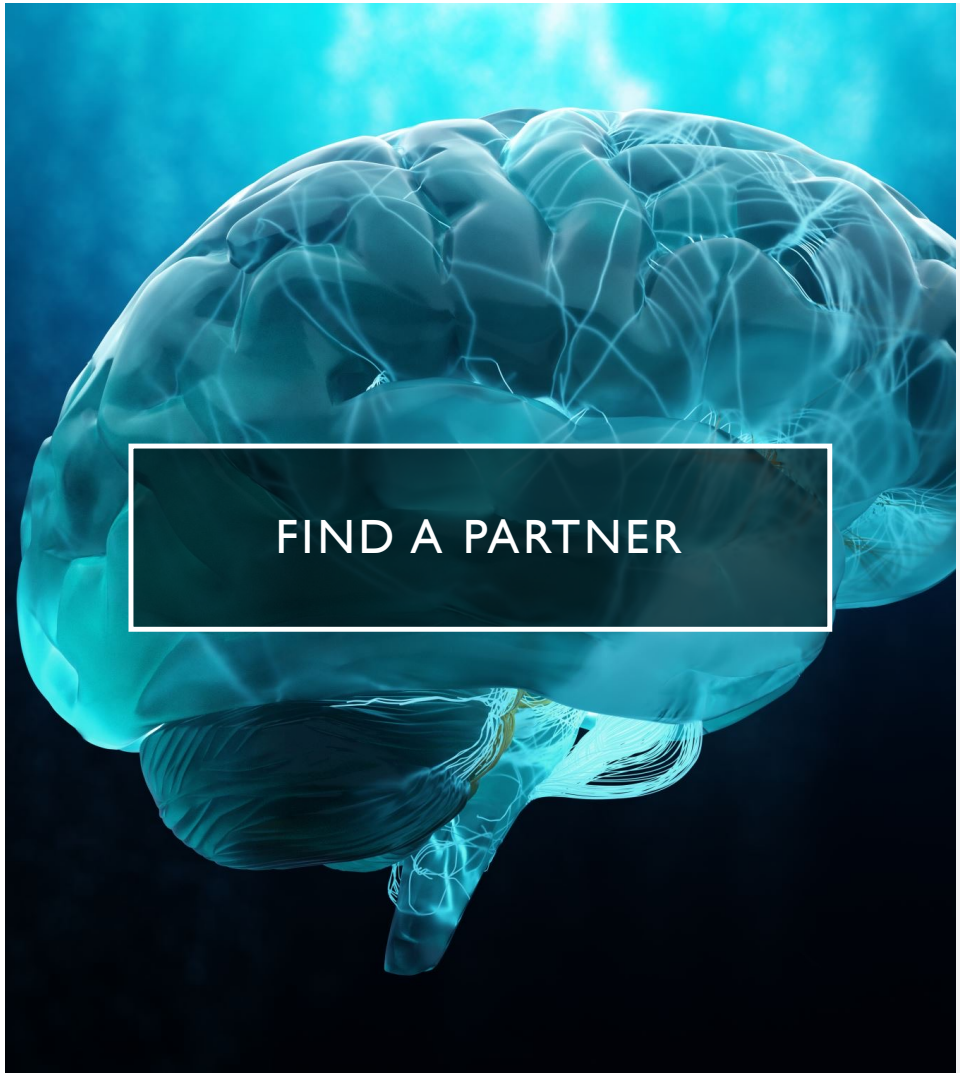
Code block at new indentation level

- Any of the code blocks associated with if, elif or else can also contain a conditional statement (an if …)
- We call these nested conditionals or nested if statements
- The nested if (inside an if code block) gets its own indented code block

```
if temperature > 20:
    if not windy:
        print ("Perfect weather, lets go to the beach!")
    else:
        print("The temperature is nice, but it is too windy.")
else:
    print("It is too cold, I'll stay home.")
```



- Complete the Basketball example
- In the WNBA if you are fouled while taking a shot and your shot does not go in, you get the equivalent number of free throws as your shot was worth (2 for a 2 pointer and 3 for a 3 pointer).
- If your shot does go in (and you were fouled) you get the appropriate amount of points plus 1 additional free throw.
- If you miss your shot and no foul occurs you get no points
- If you make your shot and no foul occurs you get the appropriate number of points



- Brain storm another real-world example where we might have complex logic or nested if statements to model it in software

A FEW NOTES ABOUT COMPARISONS



FLOATING POINT VALUES

- Usually we do not use == to compare floating point values
- Floating point numbers can be challenging to fully represent in a discrete finite binary system
- Just like 1/3 is challenging to represent in decimal, various fractions are challenging in binary (such as 0.1) as they produce long or repeating sequences
- This leads to rounding errors and makes it unreliable to compare directly two floating point numbers
- Example:

```
#how many pennies (former name for one cent coins) is $72.99?
money = 72.99
cents = money * 100
print(cents)
```

```
7298.999999999999
```

WHAT IS THE OUTPUT?

```python
# Consider the following
# what is the output

invoice = 18.58
first_payment = 18.10
second_payment = 0.48

remainder = invoice - (first_payment + second_payment)

print(f"invoice: {invoice}")
print(f"total payments: {first_payment + second_payment}")

if remainder == 0:
    print("invoice paid")
else:
    print("more money to pay")
```

WHAT IS THE OUTPUT?

Whoops! That's not right

```
# Two deposits into an account
balance = 0.10 + 0.20

print(balance)
print(balance == 0.30)
```

WHAT IS THE OUTPUT?

Whoops! That's not right!

```
# Two deposits into an account
balance = 0.10 + 0.20

print(balance)
print(balance == 0.30)
```

Output:

```
0.30000000000000004
False
```

SOLUTION 1: THRESHOLDS

This is now correct

```
# Two deposits into an account
balance = 0.10 + 0.20

print(balance)
print(balance == 0.30)

threshold = 0.0000001
print(abs(balance - 0.30) < threshold)
```

Output:

```
0.30000000000000004
False
True
```

SOLUTION 2: MATH.ISCLOSE

- The math library has a method isclose that determines if 2 floating point numbers are really close
  - Has a default tolerance of 9 decimal places but can be configured

```
math.isclose(float_variable_1, float_variable_2)
```

COMPARING FLOATS

Fix the code to avoid directly comparing floating points

```python
# Consider the following
# what is the output

invoice = 18.58
first_payment = 18.10
second_payment = 0.48

remainder = invoice - (first_payment + second_payment)

print(f"invoice: {invoice}")
print(f"total payments: {first_payment + second_payment}")

if remainder == 0:
    print("invoice paid")
else:
    print("more money to pay")
```

COMPARING FLOATS

Can use a threshold to identify the tiny value left over doesn't mean anything in our comparison.

```
# Consider the following
# what is the output

invoice = 18.58
first_payment = 18.10
second_payment = 0.48
threshold = 0.0000001

remainder = invoice - (first_payment + second_payment)

print(f"invoice: {invoice}")
print(f"total payments: {first_payment + second_payment}")
print(f"remainder: {remainder}")

if abs(remainder) < threshold:
    print("invoice paid")
else:
    print("more money to pay")
```

Output:

```
invoice: 18.58
total payments: 18.580000000000002
remainder: -3.552713678800501e-15
invoice paid
```

COMPARING FLOATS

Fix the code to avoid directly comparing floating points

```python
# Fix the example to use a threshold or math.isclose

invoice = 18.58
first_payment = 18.10
second_payment = 0.48

remainder = invoice - (first_payment + second_payment)

print(f"invoice: {invoice}")
print(f"total payments: {first_payment + second_payment}")

threshold = 0.00000001
if abs(remainder - 0) < threshold:
    print("invoice paid")
else:
    print("more money to pay")
```

```
invoice: 18.58
total payments: 18.580000000000002
invoice paid
```

COMPARING STRINGS

- Characters are represented as numbers (binary numbers)
- For example "A" is character 65,
- Strings are compared by their binary code (ordinal number)

```
cook@pop-os:~$ ascii -d
   0 NUL    16 DLE    32      48 0    64 @    80 P    96 `   112 p
   1 SOH    17 DC1    33 !    49 1    65 A    81 Q    97 a   113 q
   2 STX    18 DC2    34 "    50 2    66 B    82 R    98 b   114 r
   3 ETX    19 DC3    35 #    51 3    67 C    83 S    99 c   115 s
   4 EOT    20 DC4    36 $    52 4    68 D    84 T   100 d   116 t
   5 ENQ    21 NAK    37 %    53 5    69 E    85 U   101 e   117 u
   6 ACK    22 SYN    38 &    54 6    70 F    86 V   102 f   118 v
   7 BEL    23 ETB    39 '    55 7    71 G    87 W   103 g   119 w
   8 BS     24 CAN    40 (    56 8    72 H    88 X   104 h   120 x
   9 HT     25 EM     41 )    57 9    73 I    89 Y   105 i   121 y
  10 LF     26 SUB    42 *    58 :    74 J    90 Z   106 j   122 z
  11 VT     27 ESC    43 +    59 ;    75 K    91 [   107 k   123 {
  12 FF     28 FS     44 ,    60 <    76 L    92 \   108 l   124 |
  13 CR     29 GS     45 -    61 =    77 M    93 ]   109 m   125 }
  14 SO     30 RS     46 .    62 >    78 N    94 ^   110 n   126 ~
  15 SI     31 US     47 /    63 ?    79 O    95 _   111 o   127 DEL
```

"A" comes before "Z" which comes before "a"
(65) (90) (97)

ORD

- The python built-in function ord(character) can be used to retrieve the ordinal number for a specified character
- Example: ord("A") ←returns 65

SELF CHECK

- What is the output of the following comparisons?
- `"BBB" < "aaa"`
- `"a" == "A"`
- `"9" > "8"`

SELF CHECK

- What is the output of the following comparisons?
- `"BBB" < "aaa"` **True**
- `"a" == "A"` **False**
- `"9" > "8"` **True**

TRY AT HOME

- Write a program to ask for the user's age. For those less than 18 output "Minor" otherwise output "Age of Majority"
- Program to read an integer from the user and print whether it is zero, negative or positive
- Program to compute and compare Area of Rectangles
  - Write code to determine if 2 rectangles have the same area:
  - Rectangle 1 dimensions: 2.49 x 2.49
  - Rectangle 2 dimensions: 1.0 x 6.2001
- There is a small inconsistency in a number comparison program. What is the inconsistency, given an example input that exposes it and then fix it.